LeakLens

Water Leak Localization System

Team Guidebook

Everything about the project in one place:
what it is, every formula and calculation, the dashboard terms,
how much was trained and tested, and what each score means

B.Tech Project, Semester 1

Indian Institute of Information Technology Kottayam

Guide: Dr. Divya Sindhu Lekha

Ashish Ranjan • Nidamanuri Kedar Vaishnav • Abhinandh Rajeev • Kadthala Hrishith Reddy

October 2026

How to use this book. Read Part A to understand the project. Parts B to E explain how the data is made, how the system decides, and how it is trained and scored. Part F explains every word and number on the dashboard. Part G is for whoever runs the code. Every number in this book was read from the project's own result files. All data are simulated: no real sensors were used.

Contents

Part A: Understanding the project **3**

1 What is this project? **3**

1.1 The core idea: follow the water . . . 3

1.2 The system in one picture . . . 3

1.3 Headline results (test scenarios only, never seen in learning) . . . 3

2 The networks **3**

Part B: The data **4**

3 Scenarios **4**

3.1 What each event does (the physics) . . . 4

3.2 Sensor noise . . . 4

3.3 Unreliable nodes and hours . . . 5

Part C: Trained vs tested **5**

4 The 80/20 split **5**

Part D: How the system decides **6**

5 Step-by-step with formulas **6**

5.1 Step 1: pressure drop with persistence . . . 6

5.2 Step 2: learn the noise . . . 6

5.3 Step 3: unaccounted flow . . . 6

5.4 Step 4: fourteen numbers . . . 7

5.5 Step 5: the quiet check . . . 7

5.6 Step 6: which kind of event? . . . 7

5.7 Step 7: rank the nodes (only if the verdict is leak) . . . 7

5.8 Worked example: leak at node 15 (net3_leak_15, a test scenario) . . . 7

5.9 Fingerprints of the other kinds (Net3 test scenarios) . . . 8

Part E: Metrics and scores **8**

6 What each metric means and how it is calculated **8**

6.1 Detection: is something wrong? . . . 8

6.2 Classification: which kind of event? . . . 9

6.3 Localization: where is the leak? . . . 10

6.4 By leak type, node type and network (final method, test leaks) . . . 10

6.5 By leak size . . . 11

6.6 Unseen network (leave-one-network-out) . . . 11

Part F: The dashboard **11**

7 Screen map and every term **12**

Part G: The software **14**

8 Code, data and how to run **14**

9 Questions the team may be asked **14**

10 Honest limitations **15**

11 Next steps (Semester 2) **15**

12 Glossary **16**

Appendix A: formula cheat sheet **17**

Appendix B: code map **17**

Part A: Understanding the project

1 What is this project?

Cities lose a lot of treated water through leaking pipes (the literature reports 3% to over 50%, depending on maintenance). A pressure drop alone does not prove a leak: a hydrant opening, a pump stopping or a tank running low also lower the pressure. So the real question is:

> *When pressure and flow change in a water network, is it a leak or a normal change, and if it is a leak, which node is it?*

LeakLens answers this on simulated networks. It (1) notices that something differs from a normal day, (2) names the kind of event (leak, demand change, pump or reservoir change, tank change, blocked pipe, pipe ageing, or normal), and (3) for a leak, ranks the nodes where it may be. A web dashboard shows the answer in plain language with reasons. The full project title is *Water Leak Localization System*; LeakLens is its short name.

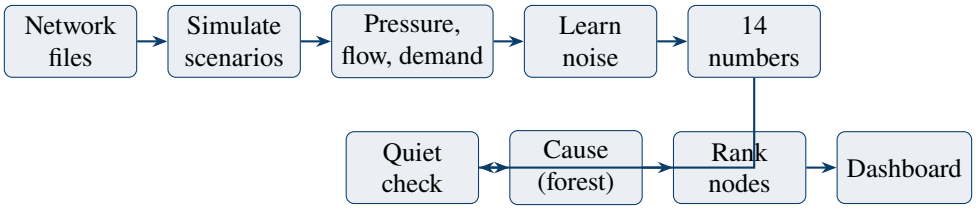
1.1 The core idea: follow the water

A junction behaves like a bank account: money in, money spent, money left. For water:

$$\underbrace{\text{water in} - \text{water out}}_{\text{net flow into the node}} = \underbrace{\text{water customers use}}_{\text{metered demand}} + \underbrace{\text{water that leaks}}_{\text{unmetered}}$$

If we know what customers use, then **water that leaves a node with no customer meter behind it is leaking**. We call this *unaccounted flow*. A leak creates a large unaccounted flow at one node. A demand change does not (the meters record it). A pump or tank change shifts pressure over a wide area but loses no water. Almost everything in the system builds on this.

1.2 The system in one picture



1.3 Headline results (test scenarios only, never seen in learning)

Question	Answer
Exact leak node ranked first (Top-1), 60 test leaks	**96.7%** (58 of 60); first version 58.3%
Real leak among the first five nodes (Top-5)	**98.3%**
False alarms on 12 normal test days	**0.0%** (fixed 0.5 m alarm: 41.7%)
Kind of event named correctly, 139 test scenarios	**82.7%** (on training scenarios: 95.6%)
Time to analyse one scenario	about 22 ms

2 The networks

Three standard EPANET benchmark networks. A network is a graph: junctions joined by pipes, plus reservoirs (sources), tanks (storage) and pumps. "Analysed" junctions exclude unreliable nodes (Section 3.3). τ_P is the learned pressure noise threshold (Section 5.2); D_{tot} is the average total customer demand in L/s.

Net	Junctions	Analysed	Pipes+	Pumps	Tanks	Reserv.	τ_P (m)	D_{tot} (L/s)	Normal runs (train/test)	Ignored node
NET1	9	9	13	1	1	1	0.498	69.4	14/4	nor
NET2	35	35	40	0	1	0	0.535	20.6	14/4	nor
NET3	92	90	119	2	3	2	0.582	690.7	14/4	601, 6

Net2 has no reservoir or pump; one junction with negative demand acts as its supply. (Pipes+ counts all links including pumps.)

Part B: The data

3 Scenarios

A **scenario** is one 24-hour simulation of a network in which one thing (or a few together) differs from a normal day. The **baseline** is the normal day. For every scenario we save, once per hour: pressure at every node, flow in every pipe, and the demand actually delivered at every node (what a customer meter reads; leak flow is not included). We also save the *true answer* (kind of event, leak node) to score the system; it is never given to the model as input.

Kind	Count	Parameters used
Leak	299	hole at one junction; areas 0.0001 / 0.0003 / 0.001 / 0.003 / 0.01 m^2 (tiny / small / medium / large / burst); start 12:00 (also 2, 8, 20); every junction in each network; noisy copies; *hidden behind* a global demand ×1.3, a hydrant, a closed pipe or a pump outage
Demand change	60	one node ×1.5 or ×0.5 from 12:00; all nodes ×1.3 or ×0.7; hydrant 0.01 m^3/s for 10:00–14:00
Pump / reservoir	54	pump off for 2, 4 or 6 h while it normally runs; reservoir head ±2, ±5, ±10 m from 12:00
Tank change	60	tank starts at 10, 20, 30, 70 or 90% full; or its pipe is closed at 10:00
Blocked pipe	48	a busy pipe closed from 10:00, or throttled (minor loss 5000)
Pipe ageing	42	every pipe's Hazen–Williams C multiplied by 0.6–0.95
Normal day	54	no change; sensor noise only (16 noisy, 1 very light, 1 clean per network)

Most non-leak scenarios also have a "noisy" twin. A leak scenario count includes the hidden-behind variants.

3.1 What each event does (the physics)

- **Leak.** Water leaves through a hole: $Q_\ell(t) = C_d\, A\, \sqrt{2\, g\, P(t)}$ with $C_d = 0.75$, hole area A, $g = 9.81$ m/s^2, pressure head P at the node. More pressure means more loss. Flow towards the node rises; pressure falls around it, most at the node. *Local and unmetered.*
- **Demand change.** Customer use at node i: $D_i(t) = D_{\text{base},i}\, m(t)\, f$, where $m(t)$ is the daily pattern and f the factor. Pressure falls where use is heavy, like a leak, but the extra flow is *metered*.
- **Pump outage / reservoir head.** Boundary conditions change: a pump off removes its added head so pressure falls over its whole area and tanks start to supply; a reservoir shifted by Δh shifts pressure everywhere by about Δh. No water is lost.
- **Tank level / isolation.** Different starting level changes the head the tank supplies; closing its pipe stops all flow through it. No water is lost.
- **Blocked pipe.** Closed pipe: flow reroutes, pressure falls downstream and may rise upstream. No water is lost.
- **Ageing.** Head loss $h_f = \dfrac{10.67\, L\, Q^{1.852}}{C^{1.852}\, d^{4.87}}$ grows as the roughness coefficient C falls: wide, smooth pressure loss, largest where flow is highest. No water is lost.

3.2 Sensor noise

Gaussian errors with fixed seeds are added to saved readings of "noisy" scenarios: *low* = 0.05 m on pressure and 0.5% on flows and demands; *high* = 0.2 m and 2%. The baseline stays clean: it is the model's expectation. Simulation uses pressure-dependent demand (the engine reduces delivered demand when pressure is low).

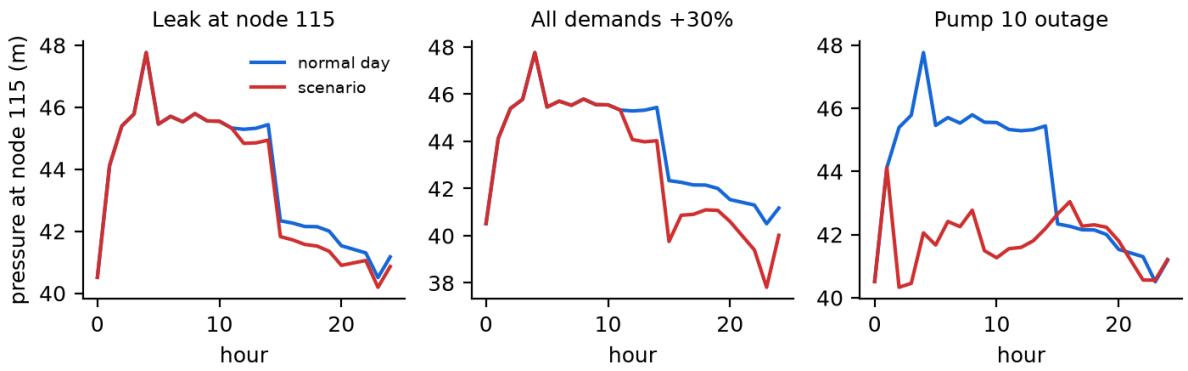


Figure 1: Pressure at one Net3 node on a normal day (blue) and in a leak, a demand surge and a pump outage (red).

3.3 Unreliable nodes and hours

A 1% demand probe is run on each network. Nodes whose pressure swings more than 5 m, and hours where 90% of nodes swing more than 0.5 m, are called ill-conditioned (pump-switching artefacts) and are ignored in all analysis: Net3 nodes **61** and **601** and hour **21:00**. Scenarios whose hydraulics did not converge were dropped (three Net3 blockage candidates).

Part C: How much was trained and how much tested

4 The 80/20 split

> Testing on scenarios the system learned from would look perfect and prove nothing. So some scenarios are kept completely aside.

All **617** scenarios were split once (random seed 42, `ml_pipeline/split.py`) into **478 for learning (training, about 80%)** and **139 for testing (about 20%)**.

Kind of event	All	Train	Test
Normal day	54	42	12
Leak	299	232	67
Demand change	60	48	12
Pump / reservoir	54	42	12
Tank change	60	48	12
Blocked pipe	48	36	12
Pipe ageing	42	30	12
Total	617	478	139

Per network, train/test counts by kind (N = normal, L = leak, D = demand, S = pump/reservoir, T = tank, B = blocked, A = ageing; Net2 has no pump or reservoir events):

Net	All	Train	Test	N	L	D	S	T	B	A
NET1	155	122	33	14/4	46/11	16/4	14/4	10/2	12/4	10/4
NET2	171	127	44	14/4	65/26	16/4	0/0	10/2	12/4	10/4
NET3	291	229	62	14/4	121/30	16/4	28/8	28/8	12/4	10/4

Rules of the split.

- **What learns from the training scenarios only:** (1) the noise thresholds (from the training normal runs, 14 per network); (2) the random forest and the decision tree.
- **What is not learned:** the leak-ranking formula (a fixed formula) and the quiet check. Their settings (weight 0.3, cut-offs) were chosen during development while looking at all scenarios, before the split, and not re-tuned afterwards.

- **What is measured on the test scenarios only:** every accuracy, precision, recall, F1, false-alarm, Top-k, hop and timing figure in this book. Training-set figures are shown only to expose over-fitting.
- **Near-copies stay together.** All leaks at the same junction (every size, start time, noise level, hidden-behind variant) go to the same side; a scenario and its noisy twin go to the same side. Otherwise a test scenario could be almost identical to a training one.
- **Stratified.** Each network and each kind of event is split about 80/20, so the test set has 12 of every non-leak kind and 60+ leaks (some leaks sit on ignored nodes and are not scored).
- The dashboard tags every scenario *Unseen test* or *Training*.

Caution. The test set is small (139 scenarios; 60 scored leaks; 12 normal days). One missed leak moves Top-1 by $1/60 \approx 1.7$ points, and the split was made once, so every percentage has a wide margin of error.

Part D: How the system decides

5 Step-by-step with formulas

Notation: $P_i^0(t)$ and $P_i^1(t)$ are pressures at node i, hour t, on the normal day and in the scenario; Q_p^0, Q_p^1 pipe flows; D_i^0, D_i^1 metered demands. Set $\mathcal{I}$ is the analysed junctions.

5.1 Step 1: pressure drop with persistence

Residual $r_i(t) = P_i^0(t) - P_i^1(t)$. To ignore one-off errors, a drop counts only if it holds for two consecutive hours:

$$\text{persistent peak}(x_i) = \max_t \ \min\{x_i(t),\ x_i(t{+}1)\}.$$

The drop of node i in noise units is $d_i = \max\{0, \text{peak}(r_i)\}/\tau_P$; the rise is $g_i = \max\{0, \text{peak}(-r_i)\}/\tau_P$.

5.2 Step 2: learn the noise

From the 14 training normal runs of each network: $\tau_P = \max\{0.1\,\text{m},\ 4\,\sigma_P\}$ where σ_P is the standard deviation of all pressure residuals over analysed nodes and hours. In the same way: $\sigma_{f,p}$ (per pipe flow), $\sigma_{D,i}$ (per node demand) and $\sigma_{a,i}$ (per node unaccounted flow, Step 3). D_{tot} is the mean over time of the total positive baseline demand. A value of "1" in noise units means *exactly at the noise limit*.

5.3 Step 3: unaccounted flow

Net inflow to node i is $\text{div}_i(t) = \sum_p \text{sign}(p,i)\, Q_p(t)$, where sign = +1 if pipe p ends at i and −1 if it starts there. Mass balance: $\text{div}_i = D_i + Q_{\ell,i}$. So the leak flow is

$$a_i(t) = \left[\text{div}_i^1(t) - \text{div}_i^0(t)\right] - \left[D_i^1(t) - D_i^0(t)\right] = Q_{\ell,i}(t).$$

Its persistent peak in noise units: $a_i = \max\{0, \text{peak}(a_i(t))\} \,/\, (4\,\sigma_{a,i})$. In noise-free data it equals the leak flow at the leak node and 0 elsewhere. When demand meters are unavailable the second bracket is dropped.

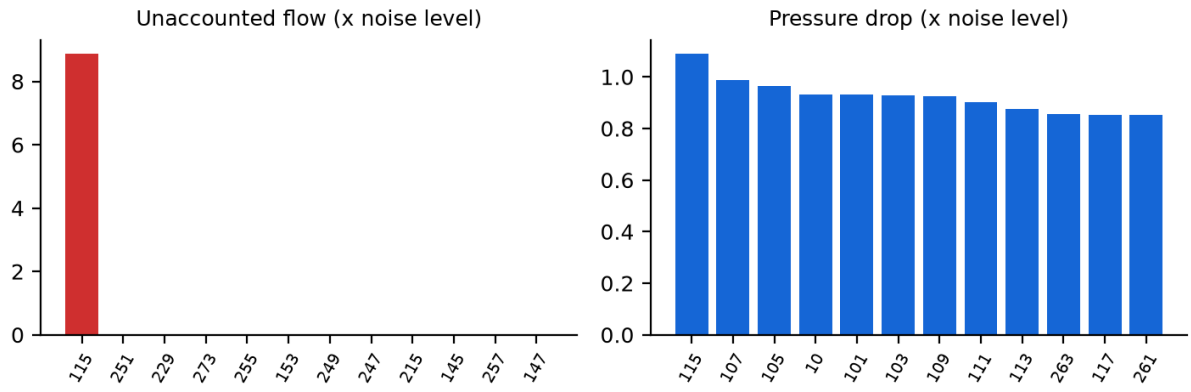


Figure 2: A leak at one Net3 node: unaccounted flow (left) is large only at the leak node; pressure drop (right) is also high at its neighbours.

5.4 Step 4: fourteen numbers

Medians over all scenarios of that kind (from `features.csv`); see Section 5.9 for individual scenarios. z_p = peak$|Q_p^1 - Q_p^0|/(4\sigma_{f,p})$; $\Delta S(t)$ is the change in total reservoir supply and $\Delta T(t)$ the change in net tank flow; k runs over pumps; z_i^D = peak$|\Delta D_i|/(4\sigma_{D,i})$.

Name	Meaning	Formula	Normal	Leak	Demand	Pump/res.	Blocked
`drop_ratio`	Largest pressure drop in noise units	$\max_i d_i$	0.20	4.25	1.11	2.61	6.82
`frac_hit`	Share of junctions with a drop above noise	mean$(d_i > 1)$	0.00	0.56	0.03	0.01	0.44
`median_ratio`	Typical (median) drop	$\text{median}_i\, d_i$	0.09	1.18	0.14	0.35	0.36
`conc_drop`	Drop concentrated at one node	$\max d_i/(\sum d_i)$	0.05	0.05	0.07	0.12	0.09
`rise_ratio`	Largest pressure rise	$\max_i g_i$	0.20	0.54	0.44	2.62	3.29
`frac_rise`	Share of junctions with a rise	mean$(g_i > 1)$	0.00	0.00	0.00	0.02	0.15
`flow_frac`	Share of pipes with changed flow	mean$(z_p > 1)$	0.00	0.60	0.35	0.47	0.65
`source_change`	Reservoir supply change / demand	$\max_t \|\Delta S(t)\|/D_{tot}$	0.01	0.01	0.01	0.37	0.01
`tank_change`	Tank net-flow change / demand	$\max_t \|\Delta T(t)\|/D_{tot}$	0.01	0.43	0.23	0.37	0.01
`pump_drop`	Pump flow lost (1 = off)	$\max(Q_k^0 - Q_k^1)/\max Q_k^0$	0.01	0.00	0.00	0.09	0.01
`unacc_ratio`	Largest unaccounted flow in noise units	$\max_i a_i$	0.25	11.16	0.06	0.08	0.07
`unacc_conc`	Unaccounted flow concentrated at one node	$\max a_i/\sum a_i$	0.07	1.00	0.02	0.01	0.02
`demand_frac`	Share of nodes with metered-demand change	mean$(z_i^D > 1)$	0.00	0.00	0.03	0.00	0.00
`demand_total`	Signed total demand change / demand	$\sum_i \Delta D_i/D_{tot}$	-0.00	-0.00	0.01	-0.00	-0.00

5.5 Step 5: the quiet check

If drop_ratio < 1, rise_ratio < 1, unacc_ratio < 1, flow_frac < 0.1 and demand_frac = 0, everything is within noise: the answer is **normal day** and nothing else runs. This is why normal days give no false alarms (and why leaks inside the noise cannot be seen).

5.6 Step 6: which kind of event?

A **random forest** (scikit-learn; 300 decision trees; each tree votes; balanced class weights so rare kinds are not ignored; seed 0; no tuning) maps the 14 numbers to a probability for each of 7 kinds. Highest probability = verdict. Confidence label: **High** ≥ 85%, **Medium** ≥ 60%, else **Low**. A depth-4 decision tree is trained too (rules in `decision_rules.txt`) for readability; it is less accurate and not used for the verdict. Training uses the 478 training rows only.

5.7 Step 7: rank the nodes (only if the verdict is leak)

For each junction, with $A_i = a_i/\max_j a_j$ (set to 0 for all nodes if $\max_j a_j < 1$) and $D_i = d_i/\max_j d_j$:

$$\textbf{score}_i = A_i + 0.3\, D_i.$$

Sort descending. First node = **Top-1**. Neighbour smoothing (+β· mean score of adjacent nodes, $\beta = 0.3$) was tested and is *off* (no gain). Error of a guess: **hops** = pipes on the shortest path to the true node; **metres** = pipe length on that path (pumps and valves count as 1 m). **Search region** = the first node plus its direct neighbours.

5.8 Worked example: leak at node 15 (net3_leak_15, a test scenario)

Network Net3, hole area 0.001 m^2 from 12:00. $\tau_P = 0.582$ m.

Hour	Normal P^0 (m)	Scenario P^1 (m)	Residual r (m)
10:00	39.74	39.74	0.00
11:00	35.98	35.98	0.00
12:00	35.91	29.22	6.69
13:00	35.88	29.20	6.68
14:00	35.91	29.23	6.68
15:00	35.44	28.81	6.63
16:00	38.77	35.92	2.85

- **Step 1:** the residual stays at about 6.68 m for several hours from 12:00; persistent peak = 6.68 m; d = 6.68/0.582 = 11.49 noise units.
- **Step 3:** unaccounted flow peak = 0.01987 m^3/s (= 19.9 L/s). Node noise σ_a = 0.000497, so $4\sigma_a$ = 0.001988 and a = 0.01987/0.001988 = 9.99 noise units (about ten times the noise).
- **Step 6:** the model gives Leak 94%, Normal day 2%, Blocked pipe 2%: verdict *leak*.
- **Step 7:** the largest a and d in the network are both at node 15, so $A = D = 1$:

Rank	Node	A	D	Score	Hops from truth
1	15	1.00	1.00	1.00+0.3×1.00 = 1.30	0
2	143	0.00	0.60	0.00+0.3×0.60 = 0.18	1
3	141	0.00	0.25	0.00+0.3×0.25 = 0.08	2
4	145	0.00	0.21	0.00+0.3×0.21 = 0.06	3

The true node is first (0 hops): a **Top-1 success**. Its neighbours have almost no unaccounted flow ($A \approx 0$) but a big pressure drop, so they only score about $0.3D$.

5.9 Fingerprints of the other kinds (Net3 test scenarios)

	Leak	Demand change	Pump outage	Tank change	Blocked pipe	
`drop_ratio`	11.49	0.03	50.59	5.71	13.33	
`frac_hit`	0.08	0.00	0.99	0.31	0.62	
`rise_ratio`	0.17	4.41	3.42	0.14	64.39	
`flow_frac`	0.30	0.12	0.84	0.34	0.87	
`source_change`	0.01	0.00	0.47	0.00	0.49	
`tank_change`	0.03	0.02	0.47	0.00	0.49	
`pump_drop`	0.00	0.00	1.00	0.00	0.43	
`unacc_ratio`	9.99	0.00	0.00	0.00	0.00	
`demand_frac`	0.00	0.01	0.00	0.00	0.00	
Verdict (confidence)	Leak (94%)	Demand change (59%)	Pump / reservoir (97%)	Tank change (84%)	Blocked pipe (90%)	Pump / rese

Read across: only the leak has a large `unacc_ratio`; the demand change is metered (`demand_frac`) with no unaccounted flow; the pump outage shows `pump_drop` = 1 and large supply/tank change; the throttled pipe shows a large pressure *rise* (`rise_ratio`) next to drops. The ageing example is a wrong answer: it is called a pump/reservoir change with only 35% confidence, because ageing and a pump event both lower pressure widely without losing water.

Part E: Metrics and scores

6 What each metric means and how it is calculated

6.1 Detection: is something wrong?

A scenario is *positive* if any abnormal event happened and *negative* if it is a normal day. Counts: TP (real problem, system says problem), FN (real problem, system says normal: a miss), FP (normal day, system says

problem: a false alarm), TN (normal day, system says normal).

$$\text{precision} = \frac{TP}{TP+FP}, \quad \text{recall} = \frac{TP}{TP+FN}, \quad F_1 = \frac{2\,\text{precision}\cdot\text{recall}}{\text{precision}+\text{recall}}, \quad \text{false-alarm rate} = \frac{FP}{FP+TN}.$$

Three detectors compared (**fixed 0.5 m**: any node's drop above 0.5 m at any hour; **learned**: persistent drop above τ_P at an analysed node; **full system**: verdict is not "normal day"). Test scenarios: 139 (12 normal).

Detector	Precision	Recall	F_1	False alarm	TP	FP	FN	TN
Fixed 0.5 m alarm	95.7%	87.4%	91.4%	41.7%	111	5	16	7
Learned pressure threshold	100.0%	77.2%	87.1%	0.0%	98	0	29	12
Full system	100.0%	93.7%	96.7%	0.0%	119	0	8	12

Calculation (full system): precision = $119/(119+0) = 100.0\%$; recall = $119/(119+8) = 93.7\%$; $F_1 = 96.7\%$; false-alarm rate = $0/(0+12) = 0.0\%$. A fixed 0.5 m alarm fires on normal noise, hence 41.7%.

6.2 Classification: which kind of event?

Confusion matrix on the 139 test scenarios (rows = truth, columns = system; a dot is 0):

Truth \ system	Normal	Leak	Demand	Pump	Tank	Blocked	Ageing	Total
Normal day	12	.	.	.	.	.	.	12
Leak	6	60	.	.	1	.	.	67
Demand change	.	.	12	.	.	.	.	12
Pump / reservoir	2	.	.	8	2	.	.	12
Tank change	.	.	.	.	8	.	4	12
Blocked pipe	.	1	.	.	1	9	1	12
Pipe ageing	.	.	.	4	2	.	6	12

Accuracy = diagonal / total = $115/139$ = **82.7%**. On the 478 training scenarios accuracy is 95.6% (the gap shows partial memorising). Per kind, with TP = diagonal, FP = column total − TP, FN = row total − TP:

Kind	TP	FP	FN	Precision	Recall	F_1
Normal day	12	8	0	12/(12+8) = 60.0%	12/(12+0) = 100.0%	75.0%
Leak	60	1	7	60/(60+1) = 98.4%	60/(60+7) = 89.6%	93.8%
Demand change	12	0	0	12/(12+0) = 100.0%	12/(12+0) = 100.0%	100.0%
Pump / reservoir	8	4	4	8/(8+4) = 66.7%	8/(8+4) = 66.7%	66.7%
Tank change	8	6	4	8/(8+6) = 57.1%	8/(8+4) = 66.7%	61.5%
Blocked pipe	9	0	3	9/(9+0) = 100.0%	9/(9+3) = 75.0%	85.7%
Pipe ageing	6	5	6	6/(6+5) = 54.5%	6/(6+6) = 50.0%	52.2%

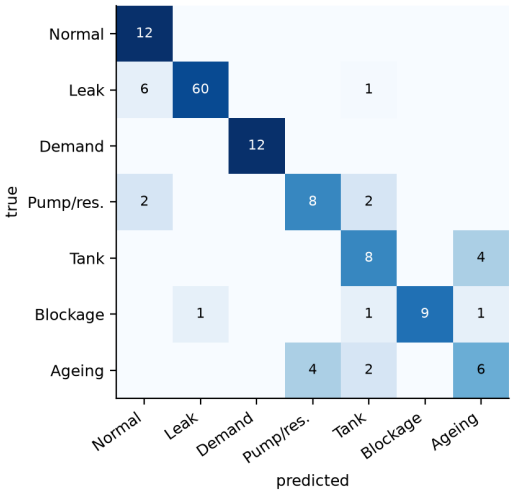


Figure 3: The confusion matrix as a picture (darker = more scenarios).

Macro-F1 = plain average of the seven F_1 values = **76.4%** (each kind counts equally, so weak small kinds pull it down). Of the leaks, 89.6% were named leaks; the 7 that were not are all at Net3 node 601, an ignored pump-outage node, and only 1.4% of non-leaks were wrongly called a leak.

6.3 Localization: where is the leak?

Rank r_s = position of the true node in the sorted list. **Top-**k = $\frac{1}{N}\sum_s \mathbf{1}[r_s \le k]$. **Mean hops** = $\frac{1}{N}\sum_s \text{hops}(\hat{n}_s, n_s)$. **Within 1 hop** = share with hops ≤ 1. **End-to-end** = share where the verdict is leak *and* $r_s = 1$. **Time** = mean seconds for ranking one scenario.

Method (60 test leaks)	Top-1	Top-3	Top-5	Mean hops	≤1 hop	Mean metres
First version	58.3%	70.0%	71.7%	3.35	58.3%	1774
Final method	96.7%	98.3%	98.3%	0.13	96.7%	60
Final + smoothing	96.7%	98.3%	98.3%	0.13	96.7%	60
Final, no demand meters	93.3%	98.3%	98.3%	0.72	93.3%	303
Pressure only	60.0%	70.0%	70.0%	3.05	61.7%	1208
Pressure only + smoothing	50.0%	63.3%	65.0%	3.18	58.3%	1244

Calculation: Top-1 = 58/60 = 96.7% (first version 35/60 = 58.3%). End-to-end 96.7%. Time about 22 ms. On the 231 training leaks the final method scores 97.8% (almost the same, because the ranking is a formula, not trained).

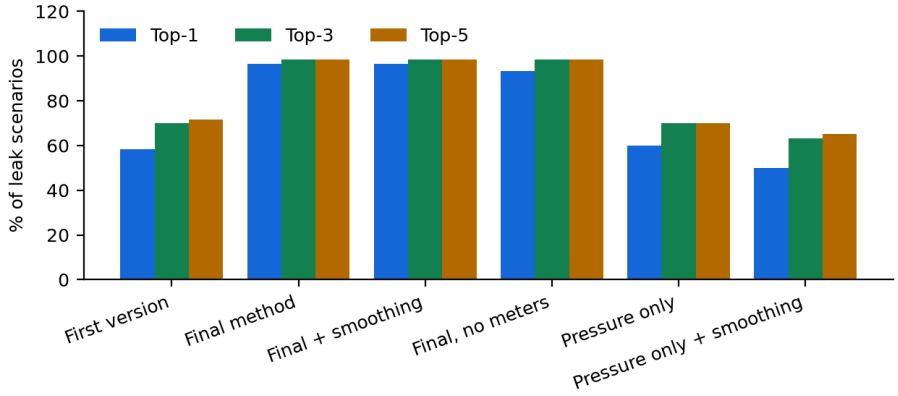


Figure 4: Top-1, Top-3 and Top-5 of each localization method on the test leaks.

What the variants show: unaccounted flow lifts Top-1 from 58.3% to 96.7%; without demand meters 93.3%; pressure alone 60.0%; neighbour smoothing 96.7% (no help).

6.4 By leak type, node type and network (final method, test leaks)

Group	Leaks	Top-1	Top-5	Mean hops
Hidden behind another event	10	100.0%	100.0%	0.00
With sensor noise	4	100.0%	100.0%	0.00
Size variations	20	90.0%	95.0%	0.40
Start-time variations	4	100.0%	100.0%	0.00
Leak at a junction	22	100.0%	100.0%	0.00
Dead End	17	100.0%	100.0%	0.00
Hub	1	100.0%	100.0%	0.00
Inline	42	95.2%	97.6%	0.19
NET1	11	100.0%	100.0%	0.00
NET2	26	92.3%	96.2%	0.31
NET3	23	100.0%	100.0%	0.00

6.5 By leak size

Area (m^2)	Test leaks	Noticed by pressure	Called a leak	Top-1	Top-5
0.0001	4	75.0%	100.0%	100.0%	100.0%
0.0003	4	100.0%	100.0%	100.0%	100.0%
0.001	4	100.0%	100.0%	100.0%	100.0%
0.003	4	100.0%	100.0%	100.0%	100.0%
0.01	4	100.0%	100.0%	50.0%	75.0%

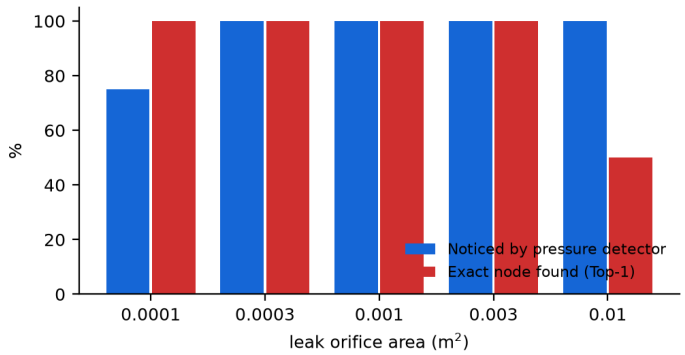


Figure 5: Detection and exact-node success by leak size.

Only four test leaks per size, so read loosely. The biggest leak collapses pressure across the whole network, so its exact node is the hardest to pick.

6.6 Unseen network (leave-one-network-out)

Train on the training scenarios of two networks, test on the third network's test scenarios:

Unseen network	Test scen.	Accuracy	Leak recall	Leak precision	Accuracy, no meters
NET1	33	48.5%	100.0%	100.0%	45.5%
NET2	44	81.8%	92.3%	100.0%	75.0%
NET3	62	54.8%	73.3%	100.0%	14.5%

Leaks carry over; telling pump, tank, blockage and ageing apart does not yet. Without per-node demand meters (all networks pooled) accuracy falls to 74.1%, and to 14.5% on an unseen Net3. This is the main Semester 2 item.

Part F: The dashboard, term by term

7 Screen map and every term

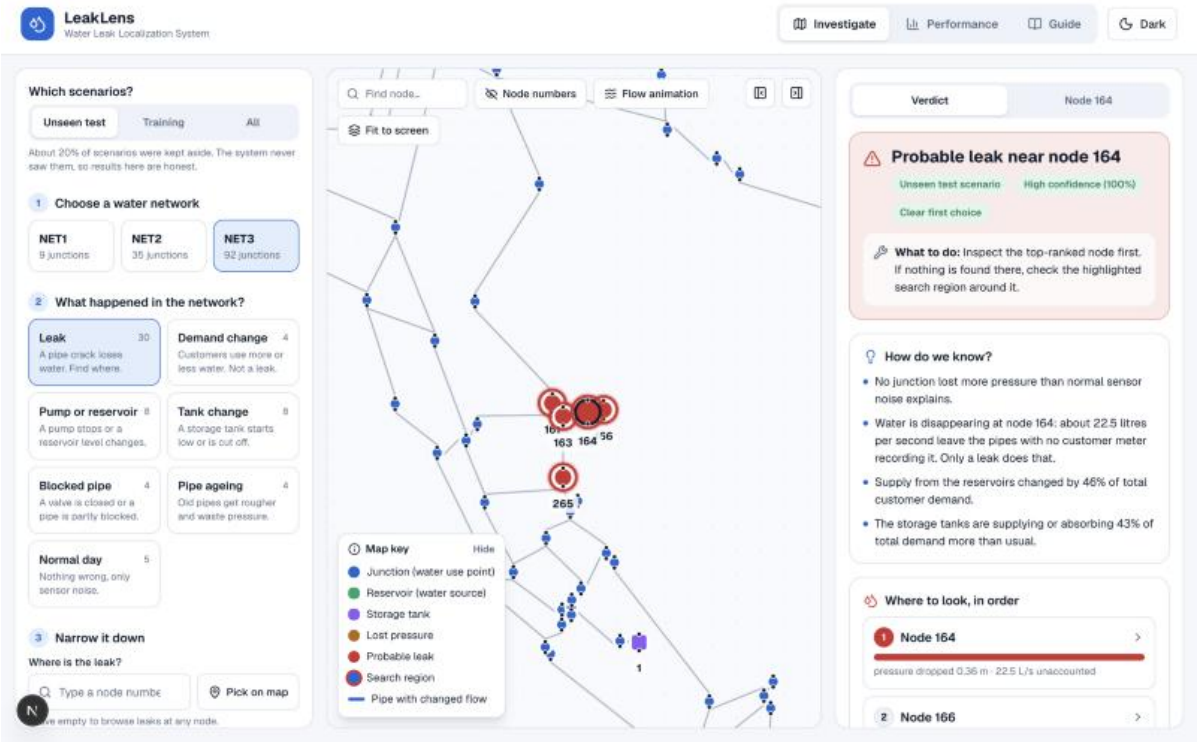


Figure 6: Investigate tab: left = choose, centre = map, right = verdict.

Header. *Investigate* (pick and analyse), *Performance* (test results), *Guide* (help and glossary), *Light/Dark* theme (saved; first visit follows the computer).

Left panel.

- **Which scenarios?** *Unseen test* (default, about 20%, never seen in learning), *Training* (the other 80%), *All*. Card counts change with this choice.
- **1 Choose a water network:** NET1 / NET2 / NET3 with junction counts.
- **2 What happened?** Seven cards (Leak, Demand change, Pump or reservoir, Tank change, Blocked pipe, Pipe ageing, Normal day); the small number is how many scenarios of that kind match the current filters.
- **3 Narrow it down.** Leaks: *Where?* (node number or *Pick on map*; leave empty for any), *How big?* (Tiny, Small, Medium, Large, Burst = 0.0001 to 0.01 m^2), *Conditions* (Clean data, Noisy sensors, Hidden behind another event, Starts at another time). Other kinds: a word search. A counter shows matches; ⟨ ⟩ step through them; *Surprise me* runs a random one. Each card shows an **Unseen test / Training** tag.
- **Customers have smart demand meters:** ticked = per-node demand is used (normal). Unticked = the unaccounted flow ignores demand, so demand changes look like leaks.
- **Analyse this scenario** runs the analysis.

Map. Circle = junction; green square = reservoir; purple square = tank; lines = pipes. After analysis: red circle = predicted leak (largest = first choice); orange = lost pressure (persistent drop above τ_P, i.e. $d_i > 1$); ring = *search region*; blue pipe = mean flow change larger than 0.01 m^3/s. Tools: *Find node*, *Node numbers*, *Flow animation*, *Fit to screen*, panel hide buttons, *Map key*.

Right panel, Verdict tab.

- **Verdict box:** for a leak "Probable leak near node X" (X = top node); otherwise the kind of event. Tags: *Unseen test scenario* / *Used in training* (with a warning that a right answer proves little), *confidence* (High/Medium/Low from the top probability, with its %), and for leaks *Clear first choice* when (2nd score)/(1st score) < 0.5, else *Close call*. *What to do* is a fixed sentence per kind.
- **How do we know?** Fixed sentences, each shown when a number passes its limit:

Sentence appears when	It says
`frac_hit`> 0 (else a “none” line)	share of junctions that lost more pressure than noise, and the biggest drop in m and its node (largest persistent drop, in metres = $d_{\max}\tau_P$)
`unacc_ratio`≥ 1	water vanishes at node X: $a_X \times 1000$ L/s unmetered
else `demand_frac`> 0 / else neither	metered demand explains the flow / all water accounted for
`pump_drop`> 0.5	a pump lost that % of flow
`source_change`> 0.2 / `tank_change`> 0.2	reservoir supply / tank flow changed by that % of total demand
`frac_rise`> 0.05	some junctions gained pressure: blocked pipe or tank
`frac_hit`> 0.5 and `unacc_ratio`< 1	loss spread over most of the network

- **Where to look, in order:** top five nodes (leaks only). Bar = score relative to the first (at least 6%). Text: “pressure dropped X m” (largest raw residual) and “Y L/s unaccounted”. Click a node to open its tab. *Search region* = first node + direct neighbours.
- **Check against the answer key:** the true event; green/red badges: *Cause correct?* and *Top-1 hit* (or “missed by n pipe hops”).
- **Technical details** (folded): probability for each kind; the 14 numbers in words; the formula **score** = $A + 0.3D$; the network’s τ_P.

Node tab. *Pressure now* = average over 24 h in the scenario; “normal” = average on the normal day; “worst drop” = largest raw residual; *Unaccounted flow* in L/s; status *Probable leak* / *Lost pressure* / *Looks normal*; chart of pressure by hour (normal blue, scenario red); the pipes touching the node.

Performance tab. Cards: leaks at exact node, real leak in first 5, false alarms, right cause (with training accuracy). Sections: finds leaks; small leaks; leak versus other causes (confusion matrix); unseen network; detailed numbers. All use test scenarios.

Guide tab. Five-step explanation, how to use the tab, kinds of situation, glossary, train/test explanation, limits. Words with a dotted underline show their definition on hover or tap.

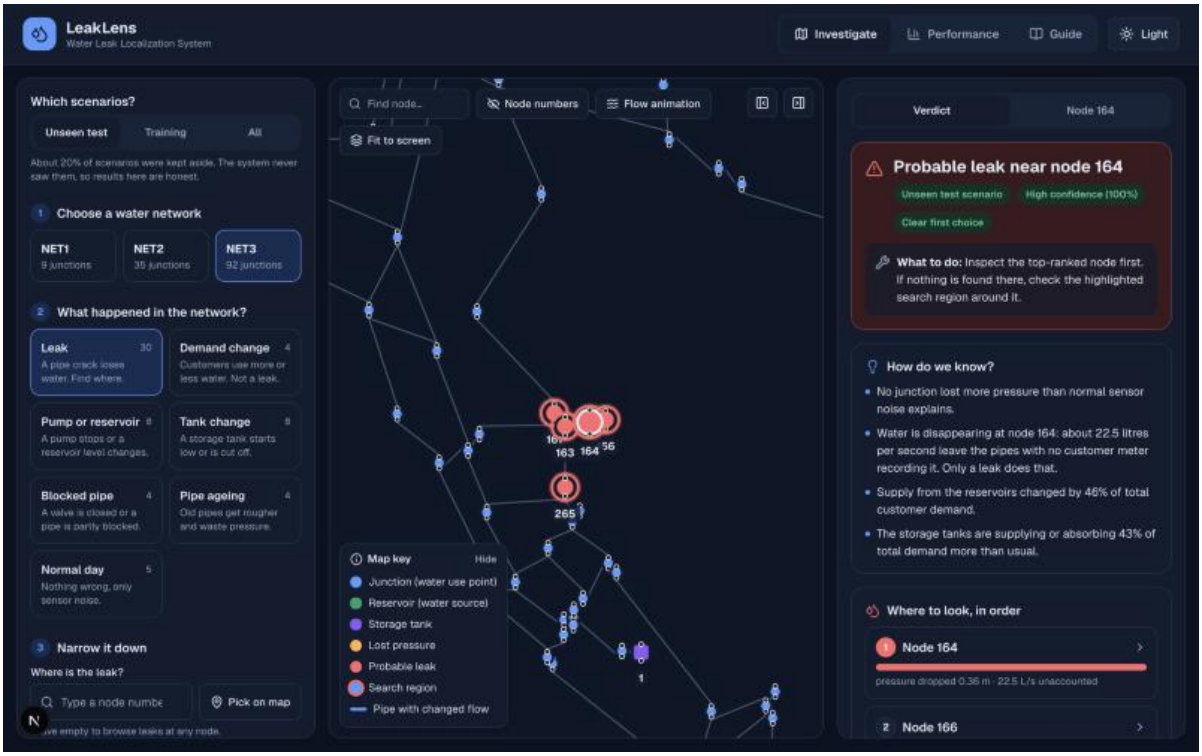


Figure 7: Dark theme. Same screen, same numbers.

Part G: The software

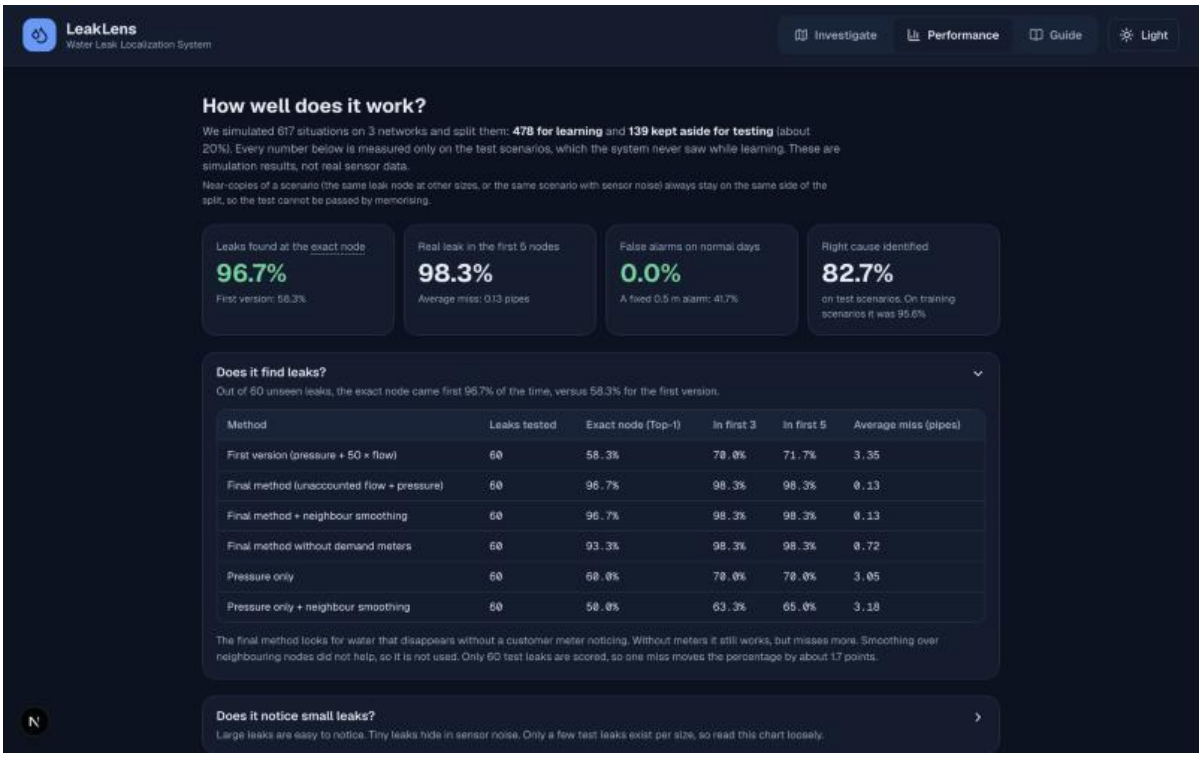


Figure 8: The Performance tab: it states the learning/testing split and reports test results only.

8 Code, data and how to run

Repository. `simulation/` (build scenarios, run EPANET/WNTR); `ml_pipeline/` (`dataset, features, detector, classifier, localizer, pipeline, split, train, evaluate`); `backend/` (FastAPI + SQLite); `frontend/` (Next.js dashboard); `data/` (networks, simulated scenarios, results); `reports/`.

Backend. At start-up it builds an SQLite table `scenarios` (id, network, name, group, event class, truth nodes, parameters, split tag) from the saved files.

Endpoint	Returns
`GET /api/networks`	networks, junction and scenario counts
`GET /api/scenarios?network=`	scenario list with parameters and the split tag
`GET /api/network?network=`	node positions and pipes (for the map)
`GET /api/analyze/{id}?use_demand_meters=true`	verdict, probabilities, 14 numbers, ranked nodes, search region, affected nodes, per-node and per-pipe numbers, split tag, answer key
`GET /api/evaluation`	the saved test results

Run order.

1. `python simulation/build_scenarios.py` (scenario list)
2. `python simulation/generate_dataset.py --force` (simulate, about 30 s)
3. `python ml_pipeline/train.py` (fit on training scenarios, writes `split.json`)
4. `python ml_pipeline/evaluate.py` (scores on test scenarios)
5. `cd backend && uvicorn main:app --port 8000`
6. `cd frontend && npm run dev` (open `http://localhost:3000`)

9 Questions the team may be asked

- **Why does it work?** Because water must balance at every node; unmetered loss at one node can only be a leak.

- **Why 80/20, and why grouped?** Testing on learned scenarios proves nothing; near-copies (same leak node at other sizes, noisy twins) would make the test too easy.
- **Why is test accuracy (82.7%) below training (95.6%)?** The forest partly memorises its 478 rows; the ranking formula is not trained, so its Top-1 barely changes (96.7% vs 97.8%).
- **Why 0% false alarms?** The threshold is learned from the network's own noise; but there are only 12 normal test days, so the true rate is not exactly zero.
- **Why is weight 0.3 used?** Chosen during development to let pressure break ties when flow data are weak. It was not re-tuned after the split.
- **What if there are no demand meters?** Demand changes look like small leaks; Top-1 falls to 93.3% and cause accuracy to 74.1%.
- **Does it work on a real network?** Unknown. Everything is simulated.

10 Honest limitations

Simulation only; small test set and a single split; ranking settings chosen before the split; per-node demand meters assumed; leaks inside sensor noise are called normal; the biggest leaks are hardest to pinpoint; pump, tank, blockage and ageing are confused with each other and do not transfer to unseen networks; Net3 nodes 61 and 601 and hour 21 are ignored; pump speed changes were not simulated (the simulator does not support them).

11 Next steps (Semester 2)

Missing and sparse sensors with graph-based sensor placement; estimating demand where nodal meters do not exist; several simultaneous leaks; cause models that transfer across networks (including graph or algorithm-informed models compared with this interpretable baseline); repeated re-splitting for tighter error bars; real or public benchmark data.

12 Glossary

Baseline	the simulated normal day used as reference
Scenario	one 24-hour simulation with a known true answer
Junction / reservoir / tank / pump	water use point / source / storage / machine adding pressure
Pressure (m)	push of the water, as metres of water column
Flow (L/s)	litres passing a point each second
Demand, demand meter	water customers draw; meter that reports it
Unaccounted flow	water leaving a node with no meter behind it = leak flow
Noise unit	measured value divided by the learned noise limit; 1 = at the limit
Persistence	a change counts only if it lasts two hours in a row
τ_P	learned pressure noise threshold ($\max\{0.1, 4\sigma_P\}$)
Quiet check	all signals within noise $\Rightarrow$ normal day
Random forest	300 small decision trees that vote on the kind of event
Confidence	the top probability from the forest (High $\geq$ 85%, Medium $\geq$ 60%)
Train / test	scenarios learned from (about 80%) / kept aside (about 20%)
Precision, recall, F_1	how often an alarm is real; how many real events are caught; their balance
False-alarm rate	share of normal days wrongly flagged
Accuracy, macro-F_1	share correct; average F_1 over kinds
Confusion matrix	table of true kind versus system answer
Top-1 / Top-5	true leak is first / within first five
Hop / metres	pipes (or pipe length) between guess and truth
Search region	top node plus its direct neighbours
Leave-one-network-out	train on two networks, test on the third

Appendix A: formula cheat sheet

Quantity	Formula
Leak flow	$Q_\ell = 0.75\, A\sqrt{2gP}$
Customer demand	$D_i(t) = D_{\text{base},i}\, m(t)\, f$
Head loss (ageing)	$h_f = 10.67\, L\, Q^{1.852}/(C^{1.852} d^{4.87})$
Residual	$r_i(t) = P_i^0(t) - P_i^1(t)$
Persistent peak	$\max_t \min\{x(t), x(t{+}1)\}$
Pressure noise threshold	$\tau_P = \max\{0.1,\ 4\sigma_P\}$
Drop / rise (noise units)	$d_i = \max\{0, \text{peak}(r_i)\}/\tau_P$; $g_i = \max\{0, \text{peak}(-r_i)\}/\tau_P$
Net inflow	$\text{div}_i(t) = \sum_p \text{sign}(p,i) Q_p(t)$
Unaccounted flow	$a_i(t) = \Delta\text{div}_i(t) - \Delta D_i(t)$; in noise units $a_i = \max\{0, \text{peak}(a_i)\}/(4\sigma_{a,i})$
Quiet check	$d_{\max} < 1,\ g_{\max} < 1,\ a_{\max} < 1,\ \text{flow_frac} < 0.1,\ \text{demand_frac} = 0 \Rightarrow$ normal
Confidence label	top probability ≥ 0.85 High, ≥ 0.60 Medium, else Low
Leak score	$\text{score}_i = a_i/\max a + 0.3\, d_i/\max d$
Clear first choice	$\text{score}_2/\text{score}_1 < 0.5$
Precision / recall	$TP/(TP+FP)$; $TP/(TP+FN)$
F_1	$2PR/(P+R)$
False-alarm rate	$FP/(FP+TN)$
Accuracy	(correct)/(all) = 115/139
Macro-F_1	mean of the seven per-kind F_1
Top-k	$\frac{1}{N}\sum_s \mathbf{1}[r_s \leq k]$
Mean hops	$\frac{1}{N}\sum_s \text{hops}(\hat{n}_s, n_s)$
Split	about 80% train / 20% test, grouped, seed 42

Appendix B: code map

File	What it does
`simulation/simulator.py`	runs one scenario in WNTR; events: leak, demand, hydrant, pump outage, reservoir head, tank level, closed or throttled pipe, roughness, combined; adds sensor noise
`simulation/build_scenarios.py`	builds the scenario list from each network's own layout; checks blockages converge
`simulation/generate_dataset.py`	simulates all scenarios, saves pressure, flow and demand CSVs, finds ignored nodes and hours
`ml_pipeline/dataset.py`	loads data and the pipe graph; incidence matrix; hops and metres
`ml_pipeline/split.py`	the grouped 80/20 split (seed 42)
`ml_pipeline/features.py`	noise learning, unaccounted flow, the 14 numbers, quiet check
`ml_pipeline/detector.py`	persistence rule and the learned pressure threshold
`ml_pipeline/classifier.py`	random forest and decision tree definitions, class names
`ml_pipeline/localizer.py`	leak score and ranking (and the first-version formula for comparison)
`ml_pipeline/pipeline.py`	one function that analyses a scenario end to end (used by the API and the evaluation)
`ml_pipeline/train.py`	fits the models on training scenarios only
`ml_pipeline/evaluate.py`	scores everything on test scenarios and writes the result files
`backend/main.py`, `catalog.py`	server start-up; builds the SQLite scenario table
`backend/routers/`	the API endpoints
`frontend/src/app/page.tsx`	dashboard layout and logic
`frontend/src/components/`	ScenarioPicker, NetworkMap, ResultView, PerformanceView, GuideView, Term, ThemeToggle
`data/results/`	`evaluation_summary.json`, `split.json`, `features.csv`, `decision_rules.txt`, per-scenario results